

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO MÔN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT
THỰC NGHIỆM CÁC GIẢI THUẬT SẮP XẾP NỘI**

HỌ VÀ TÊN: NGUYỄN PHÚC BÌNH MINH

MSSV: 25521120

LỚP: ATTN2025

TP. HỒ CHÍ MINH – THÁNG 1 NĂM 2026

1. Tạo bộ dữ liệu gồm 10 dãy, mỗi dãy khoảng 1 triệu số thực (ngẫu nhiên); dãy thứ nhất đã có thứ tự tăng dần, dãy thứ hai có thứ tự giảm dần, 8 dãy còn lại trật tự ngẫu nhiên; (5 dãy số thực, 5 dãy số nguyên):

Ở đây tôi đã có code sinh test như sau:

```
import numpy as np

MaxN = 1000000

test_case = []

test_case.append(np.sort(np.random.uniform(-1e6, 1e6, MaxN)))
test_case.append(np.sort(np.random.uniform(-1e6, 1e6, MaxN))[:,::-1])

for i in range(3):
    test_case.append(np.random.uniform(-1e6, 1e6, MaxN))

for i in range(5):
    test_case.append(np.random.randint(-1_000_000, 1_000_000, MaxN))

with open("input.txt", "w") as f:
    for a in test_case:
        f.write(" ".join(map(str, a)))
        f.write("\n")
```

Ở đây các bộ test được sinh như sau:

Test case	Kiểu	Tính chất
Test case 1	Float	Tăng dần
Test case 2	Float	Giảm dần
Test case 3	Float	Ngẫu nhiên
Test case 4	Float	Ngẫu nhiên
Test case 5	Float	Ngẫu nhiên
Test case 6	Int	Ngẫu nhiên
Test case 7	Int	Ngẫu nhiên
Test case 8	Int	Ngẫu nhiên

Test case 9	Int	Ngẫu nhiên
Test case 10	Int	Ngẫu nhiên

2. Viết các chương trình sắp xếp dãy theo các thuật toán QuickSort, HeapSort, MergeSort và chương trình gọi hàm sort của python (numpy):

Ta sẽ gộp các thuật toán vào 1 chương trình để có thể so sánh thời gian chạy như sau:

```
import sys
import time
import numpy as np

sys.setrecursionlimit(10**7)

def load_tests(filename="input.txt"):
    tests = []
    with open(filename, "r") as f:
        for line in f:
            arr = np.array(list(map(float, line.split())))
            tests.append(arr)
    return tests

def quick_sort(a, l, r):
    if l >= r:
        return
    p = a[(l + r) // 2]
    i, j = l, r
    while i <= j:
        while a[i] < p:
            i += 1
        while a[j] > p:
            j -= 1
```

```

        if i <= j:
            a[i], a[j] = a[j], a[i]
            i += 1
            j -= 1
    if l < j:
        quick_sort(a, l, j)
    if i < r:
        quick_sort(a, i, r)

def heapify(a, n, i):
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and a[l] > a[largest]:
        largest = l
    if r < n and a[r] > a[largest]:
        largest = r
    if largest != i:
        a[i], a[largest] = a[largest], a[i]
        heapify(a, n, largest)

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):
        heapify(a, n, i)
    for i in range(n - 1, 0, -1):
        a[0], a[i] = a[i], a[0]
        heapify(a, i, 0)

def merge(a, l, mid, r):
    L = a[l:mid + 1]

```

```

R = a[mid + 1:r + 1]
i = j = 0
k = 1
while i < len(L) and j < len(R):
    if L[i] <= R[j]:
        a[k] = L[i]
        i += 1
    else:
        a[k] = R[j]
        j += 1
    k += 1
while i < len(L):
    a[k] = L[i]
    i += 1
    k += 1
while j < len(R):
    a[k] = R[j]
    j += 1
    k += 1

def merge_sort(a, l, r):
    if l < r:
        mid = (l + r) // 2
        merge_sort(a, l, mid)
        merge_sort(a, mid + 1, r)
        merge(a, l, mid, r)

def numpy_sort(a):
    return np.sort(a)

tests = load_tests("input.txt")

```

```

for i, test in enumerate(tests):
    print(f"\nTest case {i + 1}")

    # QuickSort
    a = test.copy()
    start = time.perf_counter()
    quick_sort(a, 0, len(a) - 1)
    t_quick = time.perf_counter() - start
    print(f"QuickSort : {t_quick:.6f} giây")

    # HeapSort
    a = test.copy()
    start = time.perf_counter()
    heap_sort(a)
    t_heap = time.perf_counter() - start
    print(f"HeapSort : {t_heap:.6f} giây")

    # MergeSort
    a = test.copy()
    start = time.perf_counter()
    merge_sort(a, 0, len(a) - 1)
    t_merge = time.perf_counter() - start
    print(f"MergeSort : {t_merge:.6f} giây")

    # NumPy sort
    a = test.copy()
    start = time.perf_counter()
    np.sort(a)
    t_numpy = time.perf_counter() - start
    print(f"NumPy sort : {t_numpy:.6f} giây")

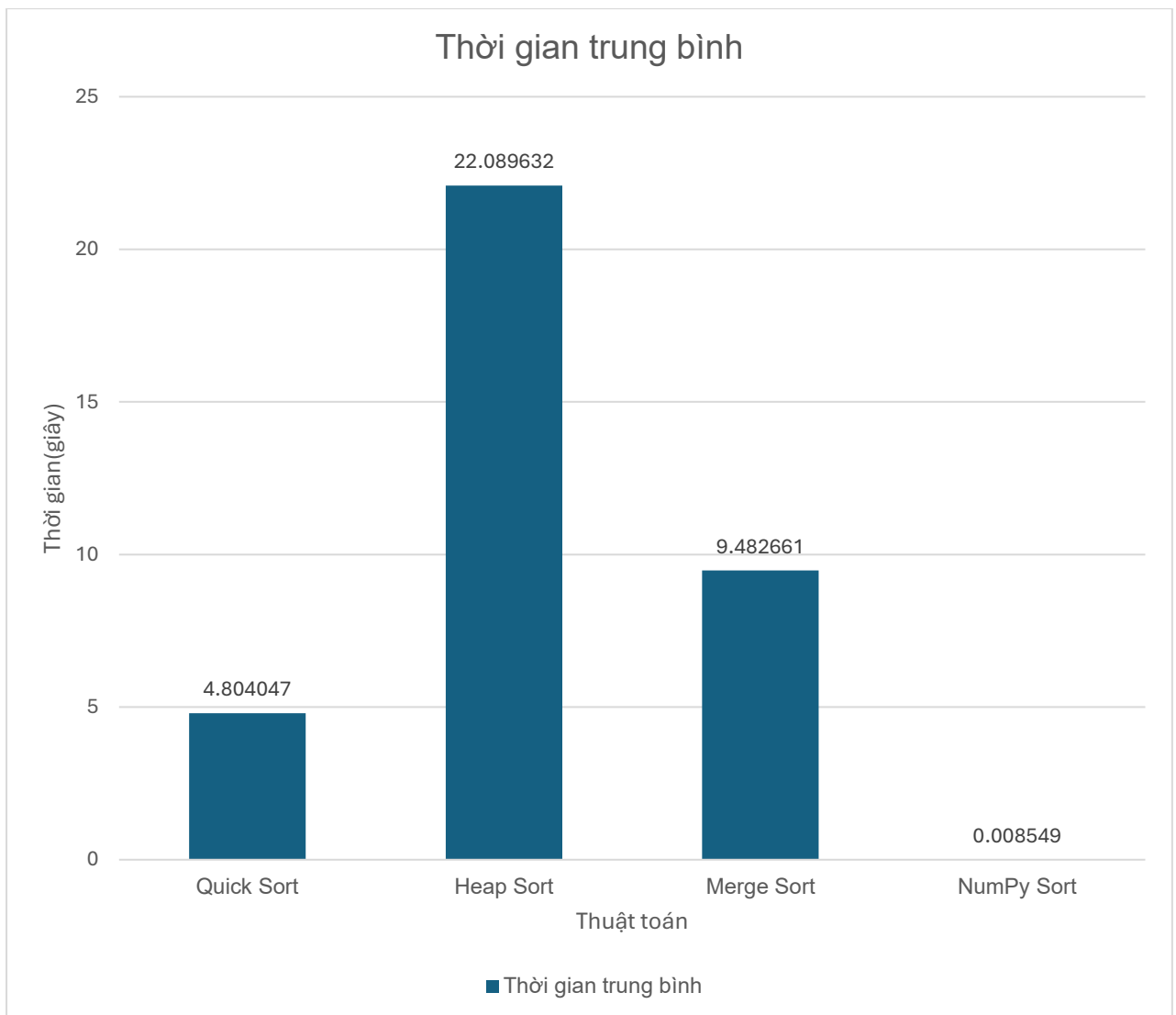
```

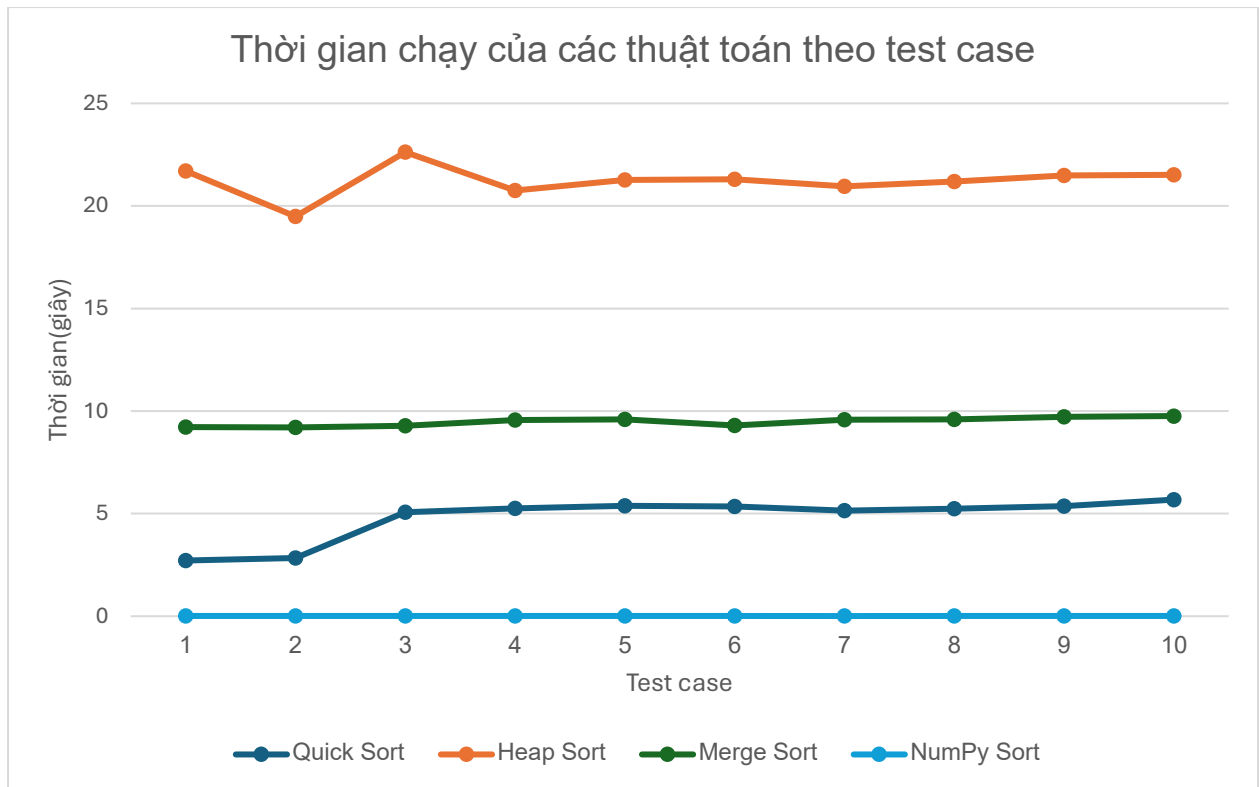
3. Chạy thử nghiệm chương trình đã viết ở trên với bộ dữ liệu đã tạo, ghi nhận thời gian thực thi từng lần thử nghiệm:

Ta có bảng dữ liệu thời gian chạy(giây) như sau:

Test case	Quick Sort	Heap Sort	Merge Sort	NumPy Sort
Test case 1	2.714408	21.705257	9.214894	0.007685
Test case 2	2.839126	19.487888	9.202712	0.009590
Test case 3	5.065082	22.621587	9.289349	0.007505
Test case 4	5.255284	20.755395	9.568648	0.008805
Test case 5	5.377306	21.271354	9.595979	0.012391
Test case 6	5.344599	21.299617	9.302346	0.007647
Test case 7	5.152585	20.949325	9.583134	0.007484
Test case 8	5.244041	21.183997	9.597115	0.008673
Test case 9	5.364935	21.488372	9.714564	0.007584
Test case 10	5.683100	21.517372	9.757864	0.008129
Average	4.804047	22.089632	9.482661	0.008549

Ta có dạng biểu đồ như sau:





Nhận xét:

Ảnh hưởng của dữ liệu đã sắp xếp:

Ở dữ liệu đã được sắp xếp tăng dần hoặc giảm dần **Quick Sort** cho thời gian thực thi thấp nhất trong các thuật toán tự cài đặt và **Quick Sort** cũng có thời gian thực thi với bộ dữ liệu đã sắp xếp là nhanh hơn đáng kể so với các dữ liệu phân bố ngẫu nhiên. Trong khi đó, thời gian chạy của **Heap Sort** và **Merge Sort** gần như không thay đổi cho thấy hai thuật toán này không phụ thuộc vào thứ tự ban đầu của dữ liệu. **NumPy Sort** thực thi cực nhanh trên cả dữ liệu đã sắp xếp chỉ khoảng 0.007–0.009 giây.

Hiệu năng trên dữ liệu sinh random:

- **Quick Sort:** thời gian dao động khoảng 5.0–5.7 giây vẫn duy trì hiệu năng tốt phù hợp cho dữ liệu lớn.
- **Merge Sort:** thời gian ổn định quanh 9.2–9.7 giây thể hiện đặc tính không phụ thuộc vào dữ liệu.
- **Heap Sort:** thời gian duy trì cao 20–22 giây ít biến động giữa các bộ test.
- **NumPy Sort:** luôn duy trì thời gian cực thấp 0.007–0.012 giây, cho thấy độ ổn định và hiệu năng vượt trội bất kể dữ liệu đã sắp xếp hay ngẫu nhiên float hay int.

Ảnh hưởng của kiểu dữ liệu:

- So sánh dữ liệu **float** và **int** cho thấy các thuật toán có thời gian thực thi không thay đổi đáng kể chứng tỏ kiểu dữ liệu không ảnh hưởng nhiều.

Độ ổn định:

- **Quick Sort** vượt trội trong các thuật toán tự cài đặt thích hợp với dữ liệu lớn và ngẫu nhiên.
- **Merge Sort** phù hợp khi cần hiệu năng ổn định.
- **Heap Sort** có thời gian cao và ít biến động không tối ưu.
- **NumPy Sort** nổi bật về hiệu năng và độ ổn định vượt xa các thuật toán tự cài đặt.

Kết luận:

- **Quick Sort** là lựa chọn hiệu quả nhất về mặt thời gian trong các thuật toán tự cài đặt.
- **Merge Sort** nên sử dụng khi cần sắp xếp ổn định.
- **Heap Sort** không tối ưu thời gian chạy quá cao.
- **NumPy Sort** hiệu năng vượt trội ổn định tuyệt đối phù hợp cho mọi loại dữ liệu lớn nên ưu tiên sử dụng trong thực tế.

LINK GITHUB:

<https://github.com/binprolayerr/DSA>